

— SPRING BOOT MICROSERVICES FUNDAMENTALS —

☆ What are Microservices?

Microservices is an architectural style where an application is built as a collection of small, independent services that communicate with each other over a network.

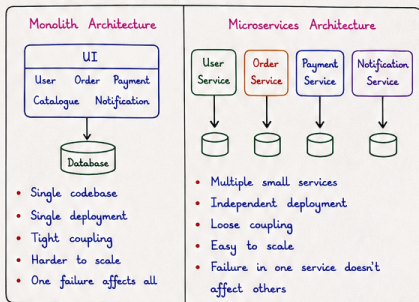
Real-world Examples

- **Netflix** (User, Catalogue, Recommendation)
- **Amazon** (Product, Order, Payment, Shipping)
- **Uber** (Ride, Driver, Payment, Notification)

☆ Why Microservices?

- ✓ Independent development & deployment
- ✓ Technology flexibility
- ✓ Easy scalability
- ✓ Better fault isolation
- ✓ High maintainability
- ✓ Continuous delivery
- ✓ Teams can work in parallel

☆ Monolith vs Microservices



☆ Advantages

- Scalability
- Independent Deployment
- Fault Isolation
- Technology Diversity
- Better Team Productivity
- Reusability of Services

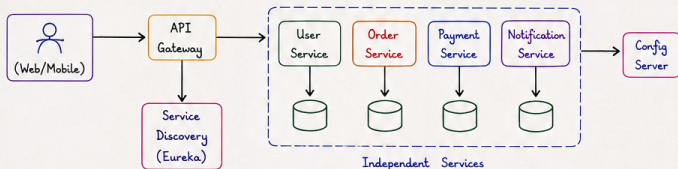
☆ Challenges

- Complexity in development
- Distributed system issues
- Network latency
- Data consistency
- Monitoring & Logging
- Deployment & management

Interview Tip

Every microservice should have its own database and should be independently deployable.

☆ Typical Microservices Architecture



Key Takeaway

Break your application into small, independent services. Each service should do one thing and do it well.

2. MICROSERVICE ARCHITECTURE

☆ What is Microservice Architecture?

Microservice architecture is an approach to developing a single application as a suite of **small services**, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API.

☆ Key Principles

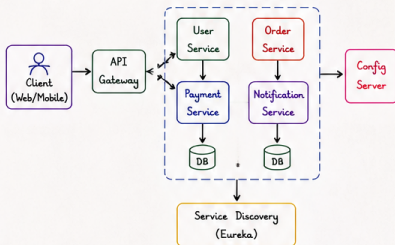
- ✓ Single Responsibility Principle
- ✓ Independent Deployment
- ✓ Decentralized Data Management
- ✓ Loose Coupling
- ✓ High Cohesion
- ✓ Fault Isolation
- ✓ Scalability

Each service is small, focused and independent.

☆ Monolith vs Microservices

Monolith	Microservices
• Single codebase • Single deployment • Shared database • Tight coupling • Harder to scale • One failure can bring down entire application	• Multiple independent services • Independent deployment • Each service has its own database • Loose coupling • Easy to scale individual services • Failure in one service doesn't affect others

☆ Typical Microservices Architecture



☆ Characteristics

- Services are independently deployable.
- Services communicate over the network.
- Each service owns its data.
- Services are fault tolerant.
- Services are designed around business capabilities.

☆ Benefits

- Independent development and deployment
- Technology freedom
- Easier scaling
- Better fault isolation
- Continuous delivery
- Team autonomy

☆ Challenges

- Complexity in development
- Distributed system issues
- Network latency
- Data consistency
- Monitoring & logging
- Deployment & management



Interview Tip

When to use Microservices?

Use microservices when your application:

- ✓ is large and complex
- ✓ requires independent scaling

- ✓ is built by multiple teams
- ✓ needs high availability

☆ Real-world Examples

NETFLIX	amazon	Uber	airbnb
• Thousands of microservices • Handles millions of users • Highly scalable • Independent deployments	• Microservices for product, order, payment, shipping • High availability • Scalable architecture	• Services for rides, payment, driver, maps, notifications • Real-time communication • Loose coupling	• Microservices for booking, search, user, payments • Global scalability • High performance



Key Takeaway

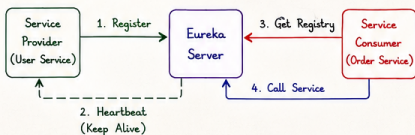
Break your application into small, independent services. Each service should be responsible for a specific business capability and should be **independently deployable and scalable**.

3. SERVICE DISCOVERY (EUREKA)

☆ What is Eureka?

Eureka is a Netflix open-source service discovery server. It allows microservices to register themselves and discover other services dynamically without hardcoding host and port.

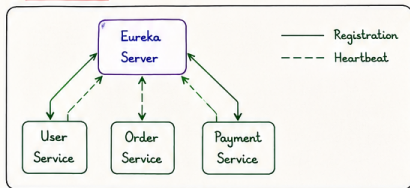
☆ How Eureka Works?



☆ Key Points

- Services register with Eureka Server.
- Eureka Server maintains the registry.
- Services send heartbeat to keep their registration alive.
- Consumers fetch the registry and call required services.
- If a service is down, Eureka removes it from the registry.

☆ Architecture



☆ Eureka Components

Component	Description
Eureka Server	Central server that maintains the service registry.
Service Provider	Microservice that registers itself with Eureka.
Service Consumer	Microservice that discovers and calls other services.
Eureka Client	Library included in each microservice to interact with Eureka Server.

☆ Benefits

- ✓ No need to hardcode IP addresses and ports.
- ✓ Automatic failover and high availability.
- ✓ Load balancing at the client side.
- ✓ Easy to scale microservices.
- ✓ Improves resilience and flexibility.

Interview Tip

Eureka is a server-side discovery. The client fetches the service list and performs load balancing.

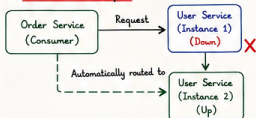
☆ Application Properties (Eureka Server)

```
server.port = 8761
eureka.client.register-with-eureka = false
eureka.client.fetch-registry = false
eureka.server.enable-self-preservation = false
```

☆ Application Properties (Eureka Client)

```
eureka.client.service-url.defaultZone = http://localhost:8761/eureka/
eureka.instance.prefer-ip-address = true
eureka.instance.instance-id = ${spring.application.name}:${random.value}
```

☆ Failover Example



☆ Common Annotations

Annotation	Used In	Purpose
@EnableEurekaServer	Eureka Server	Enables Eureka Server
@EnableDiscoveryClient	Client Service	Registers the service with Eureka
@EnableEurekaClient	Client Service	(Deprecated) Same as above



Key Takeaway

Eureka helps microservices to find and communicate with each other dynamically, making the system more resilient, scalable and flexible.

4. API GATEWAY

Page - 4

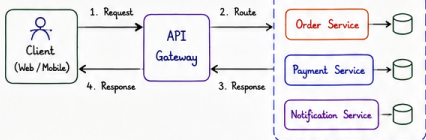
☆ What is API Gateway?

API Gateway is a **single entry point** for all client requests. It routes the requests to appropriate microservices and acts as a middleware between client and services.

☆ Key Responsibilities

- ✓ Request Routing
- ✓ Load Balancing
- ✓ Authentication & Authorization
- ✓ Rate Limiting
- ✓ Request & Response Transformation
- ✓ Logging & Monitoring
- ✓ Caching
- ✓ Global Exception Handling

☆ How API Gateway Works?



Benefits

- ✓ Single entry point
- ✓ Hides internal architecture
- ✓ Improves security
- ✓ Centralized cross-cutting concerns
- ✓ Easier to manage and monitor



Interview Tip

API Gateway is not a replacement for Load Balancer. Load Balancer works at network layer, Gateway works at application layer.

☆ Spring Cloud Gateway

- Spring Cloud Gateway is a part of Spring Cloud.
- It is built on WebFlux and Project Reactor.
- It supports non-blocking, asynchronous processing.
- It provides dynamic routing, filtering, and more.

Features

- Route predicates & filters
- Global filters
- Built-in load balancing (with Eureka)
- Security Integration
- High performance & scalable

Common Use Cases

- Routing requests to microservices
- Handling authentication
- Logging, monitoring, metrics
- Rate limiting & throttling
- Request/Response transformation

☆ Route Definition Example (application.yml)

```
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://USER-SERVICE
          predicates:
            - Path=/users/**
          → If request path matches /users/** then forward to USER-SERVICE
        - id: order-service
          uri: lb://ORDER-SERVICE
          predicates:
            - Path=/orders/**
          → If request path matches /orders/** then forward to ORDER-SERVICE
```

☆ Gateway vs Load Balancer

	API Gateway	Load Balancer
Works at	Application Layer (L7)	Network Layer (L4)
Purpose	Routing, Security, Monitoring, Transformation etc.	Distribute traffic among instances
Scope	Application specific	Infrastructure wide
Protocols	HTTP, HTTPS, WebSocket	TCP, UDP, HTTP
Example	Spring Cloud Gateway, Kong, Zuul	Nginx, HAProxy, AWS ELB

☆ Filters in API Gateway

Global Filters (applied to all routes)

- Authentication Filter
- Logging Filter
- Rate Limiting Filter
- Cors Filter
- Exception Handling Filter

Route Filters (applied to specific routes)

- AddRequestHeader
- RemoveRequestHeader
- RewritePath
- AddResponseHeader
- RequestRateLimiter



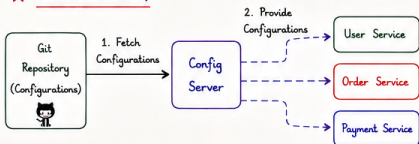
Key Takeaway

API Gateway is the front door of microservices architecture. It handles **routing**, **security** and other cross-cutting concerns in a **centralized** way.

☆ What is Config Server?

Config Server is a **centralized server** that manages external configuration for all microservices. It allows us to store configuration in **Git repository** and provides it to services at runtime.

☆ How It Works?



☆ Key Features

- ✓ Centralized configuration management
- ✓ External configuration for all environments
- ✓ Dynamic refresh without restart
- ✓ Version control with Git
- ✓ Environment-specific configurations
- ✓ Secure and consistent configuration

Benefits

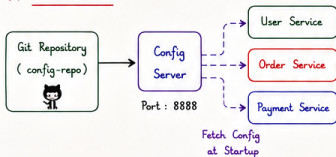
- No need to redeploy for configuration changes
- Easier environment management (Dev, Test, Prod)
- Consistent configuration across services
- Keeps sensitive data in one place
- Improves scalability and maintainability



Interview Tip

Never store sensitive information in application properties. Use Config Server with Git and encrypt sensitive data.

☆ Architecture



☆ Configuration Flow

1. Config Server connects to Git repository.
2. It loads configuration files.
3. Services request configuration at startup.
4. Config Server provides configuration.
5. Services refresh configuration on change (using `/actuator/refresh`).

☆ Sample Git Repository Structure

```

config-repo/
├── user-service/
│   ├── application.yml
│   ├── application-dev.yml
│   └── application-prod.yml
├── order-service/
│   ├── application.yml
│   ├── application-dev.yml
│   └── application-prod.yml
└── payment-service/
    ├── application.yml
    ├── application-dev.yml
    └── application-prod.yml
  
```

Each service has its own configuration files for different environments.

☆ application.yml (Config Server)

```

server:
  port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/your-username/config-repo
          search-paths: user-service, order-service, payment-service
          default-label: main
  
```

☆ application.yml (Client - Example)

```

spring:
  application:
    name: user-service
  cloud:
    config:
      uri: http://localhost:8888
      profile: dev
      label: main
  
```

Client connects to Config Server and fetches the configuration.

☆ Refresh Configuration

- When configuration changes in Git repo:
- Update the file in Git
 - Commit and push
 - Call `/actuator/refresh` endpoint in the service
 - New configuration is applied.



☆ Common Endpoints

<code>/actuator/health</code>	→ Health status
<code>/actuator/info</code>	→ Application info
<code>/actuator/env</code>	→ Environment details
<code>/actuator/refresh</code>	→ Refresh configuration
<code>/actuator/bus-refresh</code>	→ Refresh across services (with Spring Cloud Bus)



Key Takeaway

Config Server centralizes all configurations, makes environment management easier, and allows us to update configurations **without redeploying services**.

★ Why Communication ?

Microservices are **independent** and run in different processes. They need to **communicate** with each other to perform business operations.

★ Communication Types

Synchronous Communication

- Request / Response
- Real-time
- Uses HTTP / gRPC
- Examples : RestTemplate, WebClient, OpenFeign

Asynchronous Communication

- Event-driven
- Not real-time
- Uses Message Brokers
- Examples : Kafka, RabbitMQ

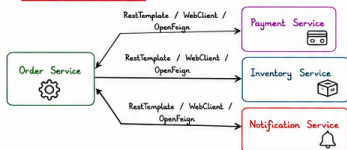
★ Synchronous Communication Approaches

1. RestTemplate		2. WebClient		3. OpenFeign	
• Provided by Spring. • Simple and easy to use. • Blocking I/O.		• Part of Spring WebFlux. • Non-blocking and reactive. • Better for high concurrency.		• Declarative REST client. • Built on top of Feign. • Easy and clean.	
Pros	Cons	Pros	Cons	Pros	Cons
• Simple API • Easy to integrate • Widely used	• Blocking • Less performant • Not recommended for high scale	• Non-blocking • High performance • Scalable	• Complex API • Steeper learning curve	• Very easy to use • Declarative • Less boilerplate	• Less control • Not good for streaming

★ Comparison

Approach	Type	Blocking	Performance	Ease of Use	Best For
RestTemplate	Synchronous	Yes (Blocking)	Medium	Easy	Simple applications
WebClient	Synchronous (Reactive)	No (Non-blocking)	High	Complex	High concurrency apps
OpenFeign	Synchronous	Yes (Blocking)	Medium	Very Easy	Microservices communication

★ Example Architecture



★ Best Practices

- ✓ Use OpenFeign for simple REST calls.
- ✓ Use WebClient for high-concurrency & reactive systems.
- ✓ Always handle timeouts and retries.
- ✓ Use Circuit Breaker for resilience.
- ✓ Use DTOs for communication, not Entities.
- ✓ Avoid tight coupling between services.
- ✓ Version your APIs.

★ Example - Using OpenFeign

```

// 1. Add Dependency
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

// 2. Enable Feign Clients
@EnableFeignClients
@SpringBootApplication
public class OrderServiceApplication { }

// 3. Create Feign Client Interface
@FeignClient(name = "payment-service")
public interface PaymentClient {

    @GetMapping("/api/payments/{id}")
    PaymentResponse getPayment(@PathVariable Long id);
}
  
```

★ Example - Using WebClient

```

WebClient webClient = WebClient.builder()
    .baseUrl("http://payment-service")
    .build();

PaymentResponse response = webClient.get()
    .uri("/api/payments/{id}", id)
    .retrieve()
    .bodyToMono(PaymentResponse.class)
    .block();
  
```

★ Timeouts & Retries (Example)

```

// With Resilience4j
@Retry(name = "paymentService", fallbackMethod = "fallback")
@CircuitBreaker(name = "paymentService")
public PaymentResponse getPayment(Long id) {
    // call payment service
}

public PaymentResponse fallback(Long id, Exception ex) {
    // return fallback response
    return new PaymentResponse("FAILED");
}
  
```



Key Takeaway

Choose the right communication approach based on your use case.

OpenFeign for simplicity, **WebClient** for performance, and always design for **resilience**.

7. FAULT TOLERANCE & RESILIENCE

Page - 7

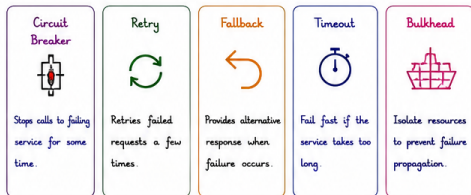
☆ Why do we need Fault Tolerance?

In distributed systems, failures are inevitable. Fault tolerance prevents **cascading failures** and ensures system reliability.

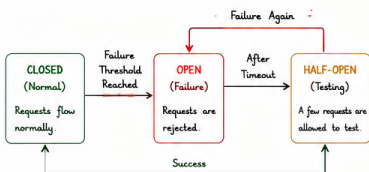
Goals

- ✓ Prevent system failures
- ✓ Improve availability
- ✓ Graceful degradation
- ✓ Quick recovery

☆ Common Resilience Patterns



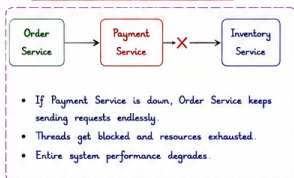
☆ Circuit Breaker - How It Works?



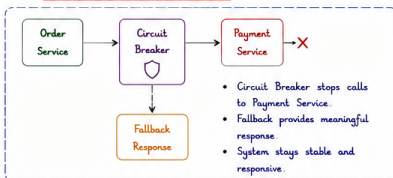
☆ Circuit Breaker States

State	Description
CLOSED	Everything is normal. Requests are allowed.
OPEN	Too many failures. All requests are blocked for a specific time.
HALF-OPEN	Allow limited requests. If successful → CLOSED else → OPEN again.

☆ Example - Without Circuit Breaker



☆ Example - With Circuit Breaker



☆ Resilience4j (Spring Boot)

- Lightweight and easy to use.
- Provides production-ready resilience patterns.
- Key Modules:
 - ✓ CircuitBreaker
 - ✓ Retry
 - ✓ RateLimiter
 - ✓ TimeLimiter
 - ✓ Bulkhead

☆ Common Annotations (Resilience4j)

Annotation	Purpose
@CircuitBreaker	Apply circuit breaker to a method.
@Retry	Retry failed calls automatically.
@TimeLimiter	Set timeout for method execution.
@Bulkhead	Limit concurrent calls.
@Fallback	Define fallback method.

☆ Example - @CircuitBreaker

```
@CircuitBreaker(name = "paymentService",
    fallbackMethod = "paymentFallback")
public String makePayment() {
    return restTemplate.getForObject(
        "http://payment-service/pay", String.class);
}

public String paymentFallback(Exception ex) {
    return "Payment service is unavailable.
    Please try again later.";
}
```

☆ Best Practices

- ✓ Use Circuit Breaker for external dependencies.
- ✓ Always provide meaningful fallback response.
- ✓ Set proper timeout and retry policies.
- ✓ Monitor failures and tune thresholds.
- ✓ Use Bulkhead to isolate critical resources.



Interview Tip

Circuit Breaker protects the system from repeated failures and gives it time to recover.

☆ Key Takeaway

Resilience patterns help us build fault-tolerant and **highly available** microservices that can handle failures gracefully.



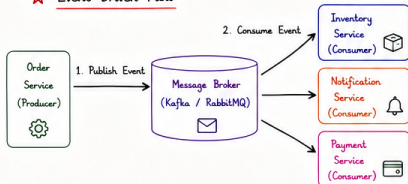
8. EVENT-DRIVEN ARCHITECTURE

Page - 8

☆ What is Event-Driven Architecture?

In event-driven architecture, services communicate by producing and consuming events. It enables **loose coupling**, **high scalability** and real-time data processing.

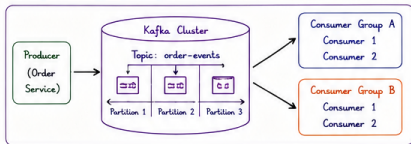
☆ Event-Driven Flow



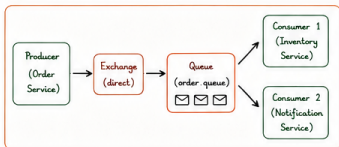
☆ Key Concepts

- **Producer** : Generates events.
- **Broker** : Receives, stores and distributes events.
- **Consumer** : Subscribes and processes events.
- **Topic (Kafka)** : Named stream of events.
- **Queue (RabbitMQ)** : Messages are distributed to specific consumers.

☆ Kafka Architecture



☆ RabbitMQ Architecture



☆ Kafka vs RabbitMQ

Feature	Kafka	RabbitMQ
Model	Publish / Subscribe	Queue (Point-to-Point or Pub/Sub)
Storage	Log-based (Disks)	In-Memory
Throughput	High	Moderate
Ordering	Per Partition	Per Queue
Scalability	Very High	High
Use Case	Real-time streaming, event sourcing	Task queues, reliable messaging

☆ When to Use?

Use Kafka When

- High throughput is needed.
- Real-time data streaming.
- Large scale event processing.
- Event sourcing & analytics.

Use RabbitMQ When

- Reliable message delivery.
- Task queues and workflows.
- Lower latency required.
- Simple and lightweight systems.

☆ Sample Event Structure (JSON)

```
{
  "eventId": "e12345",
  "eventType": "OrderCreated",
  "source": "OrderService",
  "timestamp": "2024-05-12T10:30:00Z",
  "data": {
    "orderId": "o1001",
    "customerId": "c200",
    "amount": 1500.00
  }
}
```

☆ Kafka Producer Example (Spring Boot)

@Autowired

```
private KafkaTemplate<String, OrderEvent> kafkaTemplate;
```

```
public void publishOrderEvent(OrderEvent event) {
    kafkaTemplate.send("order-events", event.getOrderId(), event);
}
```

☆ Kafka Consumer Example (Spring Boot)

```
@KafkaListener(topics = "order-events", groupId = "inventory-group")
public void consumeOrderEvent(OrderEvent event) {
    // process event
    System.out.println("Received Order Event: " + event.getOrderId());
}
```

☆ Best Practices

- ✓ Design events using clear contracts.
- ✓ Keep events immutable.
- ✓ Use meaningful event names.
- ✓ Handle failures and retries.
- ✓ Ensure idempotent consumers.
- ✓ Monitor lag and throughput.



Interview Tip

Event-driven systems improve scalability and decouple services. Always choose the right messaging system based on your use case.



Key Takeaway

Events help microservices communicate asynchronously, making the system more **resilient**, **scalable** and **flexible**.

9. DOCKER & KUBERNETES

Page - 9

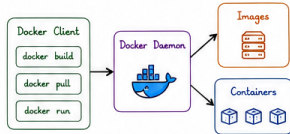
☆ What is Docker ?

Docker is a platform to develop, ship and run applications in containers. It ensures **consistency** across different environments.

☆ Key Docker Concepts

- **Image** : Read-only template with application and dependencies.
- **Container** : Runnable instance of an image.
- **Dockerfile** : Instructions to build an image.
- **Docker Hub** : Registry to store and share images.

☆ Docker Architecture



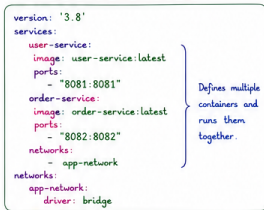
☆ Docker Flow



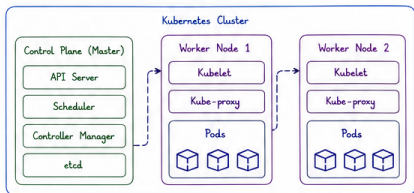
☆ Docker vs Virtual Machine

Feature	Docker	Virtual Machine
Architecture	OS Level Virtualization	Hardware Virtualization
Size	Lightweight (MBs)	Heavy (GBs)
Startup Time	Seconds	Minutes
Resource Usage	Less	High
Isolation	Process-level	Full OS-level
Best For	Microservices	Legacy Applications

☆ Docker Compose (Example)



☆ Kubernetes Architecture



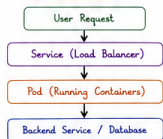
☆ Kubernetes Key Concepts

- **Pod** : Smallest deployable unit, runs one or more containers.
- **Deployment** : Manages replica sets and rolling updates.
- **Service** : Stable endpoint for accessing pods.
- **ConfigMap** : External configuration management.
- **Secret** : Store sensitive information.
- **Namespace** : Logical isolation within a cluster.

☆ Kubernetes Objects



☆ Kubernetes Flow



☆ Monitoring & Logging

- **Prometheus** : Monitoring & alerting
- **Grafana** : Visualization dashboards
- **ELK Stack** : Logs (Elasticsearch, Logstash, Kibana)
- **Zipkin / Jaeger** : Distributed tracing

☆ Benefits

- ✓ Portability
- ✓ Scalability
- ✓ High Availability
- ✓ Self-healing
- ✓ Rolling Updates
- ✓ Resource Optimization

☆ Best Practices

- ✓ Use small Docker images (Alpine).
- ✓ One process per container.
- ✓ Use .dockerignore file.
- ✓ Use liveness & readiness probes.
- ✓ Use resource limits & requests.
- ✓ Keep containers stateless.



Key Takeaway

Docker packages your application. Kubernetes runs and manages your containers at scale. Together they make microservices **reliable**, **scalable** and **production-ready**.

10 Spring Boot Microservices Interview Cheat Sheet

QUICK COMPARISONS

1 Monolith vs Microservices

	Monolith	Microservices
	Single deployable application	Multiple small services
	Tightly coupled	Loosely coupled
	Harder to scale	Easier to scale (per service)
	Single technology stack	Polyglot technology stack
	Slower development as app grows	Faster development & independent deployment
	Single point of failure	Failure isolated to specific service
	Simpler to deploy	Complex to deploy & manage

2 Eureka vs API Gateway

	Eureka (Service Discovery)	API Gateway
	Service registry & discovery	Single entry point for clients
	Services register with Eureka	Routes requests to services
	Client side or server side discovery	Centralized request routing
	No request filtering or aggregation	Can filter, aggregate, transform requests
	Runs inside the network	Exposed to outside clients
	Example: Netflix Eureka	Examples: Spring Cloud Gateway, Kong, Zuul

3 REST vs Kafka

	REST	Kafka
	Synchronous request-response	Asynchronous messaging
	HTTP/HTTPS based	TCP based
	Point-to-point (Direct call)	Publish-Subscribe (Topic based)
	Lower throughput	High throughput
	Tight coupling	Loose coupling
	Use for: CRUD, real-time operations	Use for: Event streaming, logs, analytics

4 Kafka vs RabbitMQ

	Kafka	RabbitMQ
	Distributed streaming platform	Message broker
	Topic based	Queue based
	High throughput, writes sequentially	Lower throughput, messages routed
	Horizontally scalable	Scales with clustering & queues
	Stores messages (log)	Messages removed after consumption (by default)
	Use for: Streaming, event sourcing, big data	Use for: Task queues, work distribution, RPC

5 RestTemplate vs WebClient vs OpenFeign

		RestTemplate	WebClient	OpenFeign
	Type	Synchronous (Blocking)	Asynchronous (Non-blocking)	Declarative HTTP Client
	Programming Model	Imperative	Reactive	Declarative
	Performance	Lower (Blocking I/O)	Higher (Non-blocking I/O)	Lower (Blocking, but simpler code)
	Ease of Use	Easy	Moderate	Very Easy
	Best For	Simple apps, quick integration	High performance, reactive systems	Simplify REST calls with less code

6 Docker vs Kubernetes

	Docker	Kubernetes
	Container platform	Container orchestration platform
	Build, ship & run containers	Deploy, scale & manage containers
	Single host	Multi-host (cluster)
	Manual scaling	Auto scaling & self-healing
	docker run / docker-compose	kubectl, YAML manifests
	Developer focused	Operations / DevOps focused

8 Retry vs Circuit Breaker

	Retry	Circuit Breaker
	Retries failed requests	Stops calling failing service
	Helps with temporary failures	Prevents cascading failures
	Increases load if overused	Fast failure, protects system
	Config: max attempts, backoff	Config: failure threshold, timeout, half-open
	Used with	Used for resilience pattern

7 Config Server vs application.properties

	Config Server	application.properties
	Centralized external configuration	Local configuration file
	Dynamic, can refresh at runtime	Requires restart to apply changes
	Versioned in Git	Exists in each service
	Supports multiple environments	Environment specific files
	Secure & manageable	Harder to manage at scale
	Use for microservices	Ok for small/simple applications

9 Synchronous vs Asynchronous

	Synchronous	Asynchronous
	Blocking, waits for response	Non-blocking, no need to wait
	Real-time communication	Eventual consistency
	Tight coupling	Loose coupling
	Lower throughput	Higher throughput
	Examples: REST, DB call	Examples: Kafka, RabbitMQ

10 Scaling vs Load Balancing

	Scaling (Vertical / Horizontal)	Load Balancing
	Increase resources (CPU/RAM) or add more instances	Distribute traffic across available instances
	Improves capacity	Improves availability & performance
	Can be vertical (bigger server) or horizontal (more servers)	Works at request level
	Example: Add more pod replicas	Example: Nginx, Spring Cloud LoadBalancer



Tips: Understand the trade-offs and use the right tool for the right problem.



Focus on real-world use cases and hands-on practice.